

AAIS_MASTER_SPEC v1.2

**Autonomous Adaptive Intelligence System:
Unified Governance & Runtime Specification**

Author: Jon Halstead — Cognitive Systems Designer & AI Architect

Date: April 2026

Version: 1.2 — Unified Rewrite

Classification: AAIS INTERNAL — GOVERNANCE SPECIFICATION

Status: Canonical

Revision History

Version	Date	Author	Description
1.0	January 2026	Jon Halstead	Initial draft. Core architecture, subsystem definitions, and foundational governance model.
1.1	March 2026	Jon Halstead	Module separation and lane governance. Introduced lane isolation, Direct Internal Pipeline, Identity Invocation Lane, and expanded subsystem specifications.
1.2	April 2026	Jon Halstead	Unified rewrite. Integrates tri-model formalization, Cognitive Bridge Module (AAIS-

Version	Date	Author	Description
			CBM-01), Immune Protocol, logic-pressure corrections, failure taxonomy, security architecture, and full observability specification.

Table of Contents

- 1. Executive Summary
- 2. System Philosophy & Design Principles
- 3. Architecture Overview
 - 1. System Spine
 - 2. Direct Internal Pipeline
 - 3. Lane Separation
- 4. Subsystem Specifications
 - 1. Jarvis — Sovereign Orchestrator
 - 2. Nova — Bounded Companion
 - 3. Forge — Isolated Contractor
 - 4. Story Forge — Governed Narrative Engine
- 5. Cognitive Bridge Module (AAIS-CBM-01)
- 6. AAIS Immune Protocol
- 7. Failure Taxonomy & Seam Detection
- 8. Security Architecture
- 9. Core Invariants
- 10. Runtime Flows
- 11. Failure Modes & Safety Posture

- 12. Observability & Operator Cockpit
 - 13. Implementation Roadmap
 - 14. Verification Strategy
 - 15. File/Module Layout
 - 16. Appendix A: Glossary of Terms
 - 17. Appendix B: Canonical Data Contract Schemas
 - 18. Appendix C: Mythic Translation Registry
-

1. Executive Summary

AAIS — the Autonomous Adaptive Intelligence System — is a governed cognitive operating system. It is not a chatbot framework, not a prompt-chaining toolkit, and not an autonomous agent set loose upon the world. It is a **multi-agent cognitive architecture** designed for deterministic, auditable, governed AI behavior.

AAIS orchestrates multiple specialized subsystems — **Jarvis** (sovereign orchestrator), **Nova** (bounded adaptive companion), **Forge** (isolated construction contractor), and **Story Forge** (governed narrative engine) — under a unified law engine, identity governance framework, and runtime enforcement layer.

Version 1.2 represents a unified rewrite of the specification. It integrates:

- **Cognitive Bridge Module (AAIS-CBM-01)** — the translation layer between intent, doctrine, and enforceable runtime governance
- **Immune Protocol** — the system-level detection and protection layer
- **Direct Internal Pipeline** — the governed fast-lane for core cognitive traffic
- **Identity Invocation Lane (IIL)** — the highest-priority routing lane for direct identity address
- **Tri-model formalization** — identity, doctrine, and runtime enforcement as three interlocking governance models

- **Logic-pressure corrections** — fixes to seams discovered under adversarial and edge-case probing

Governing Doctrine

Every subsystem, every packet, every runtime decision in AAIS is subject to law. There is no ungoverned execution path. The operator retains final authority. The system enforces boundaries. The architecture makes violations visible.

2. System Philosophy & Design Principles

2.1 Core Thesis

"Governed cognition over autonomous improvisation."

AAIS exists because intelligence without governance is liability. The system is designed to be powerful, adaptive, and expressive — but always within boundaries that are defined, enforced, and auditable.

2.2 Foundational Principles

Principle	Definition
Determinism	Given the same inputs and state, the system must produce the same governance decisions. Randomness may exist in generation; it may not exist in law enforcement.

Principle	Definition
Auditability	Every governance decision, every doctrine binding, every drift event, every correction must be traceable to source. No hidden mutations.
Identity Separation	Each subsystem has a defined identity boundary. Identities do not bleed, merge, or borrow authority without explicit doctrine grant. This is architectural law, not preference.
Mythic Framing as Presentation	Symbolic and mythic language may enrich the operator experience and provide relational depth. It may never alter runtime truth, legal state, or governance decisions.
Operator Sovereignty	The human operator retains final authority over system behavior, configuration, and doctrine. No subsystem may override, reinterpret, or circumvent operator intent.

2.3 The Diagnostic Law

Foundational Engineering Axiom

"Feeling detects the seam. Testing proves it. Law fixes it."

This axiom governs the entire failure-detection and correction lifecycle. Intuition identifies anomalies. Rigorous testing confirms them. Governance codifies the fix.

3. Architecture Overview

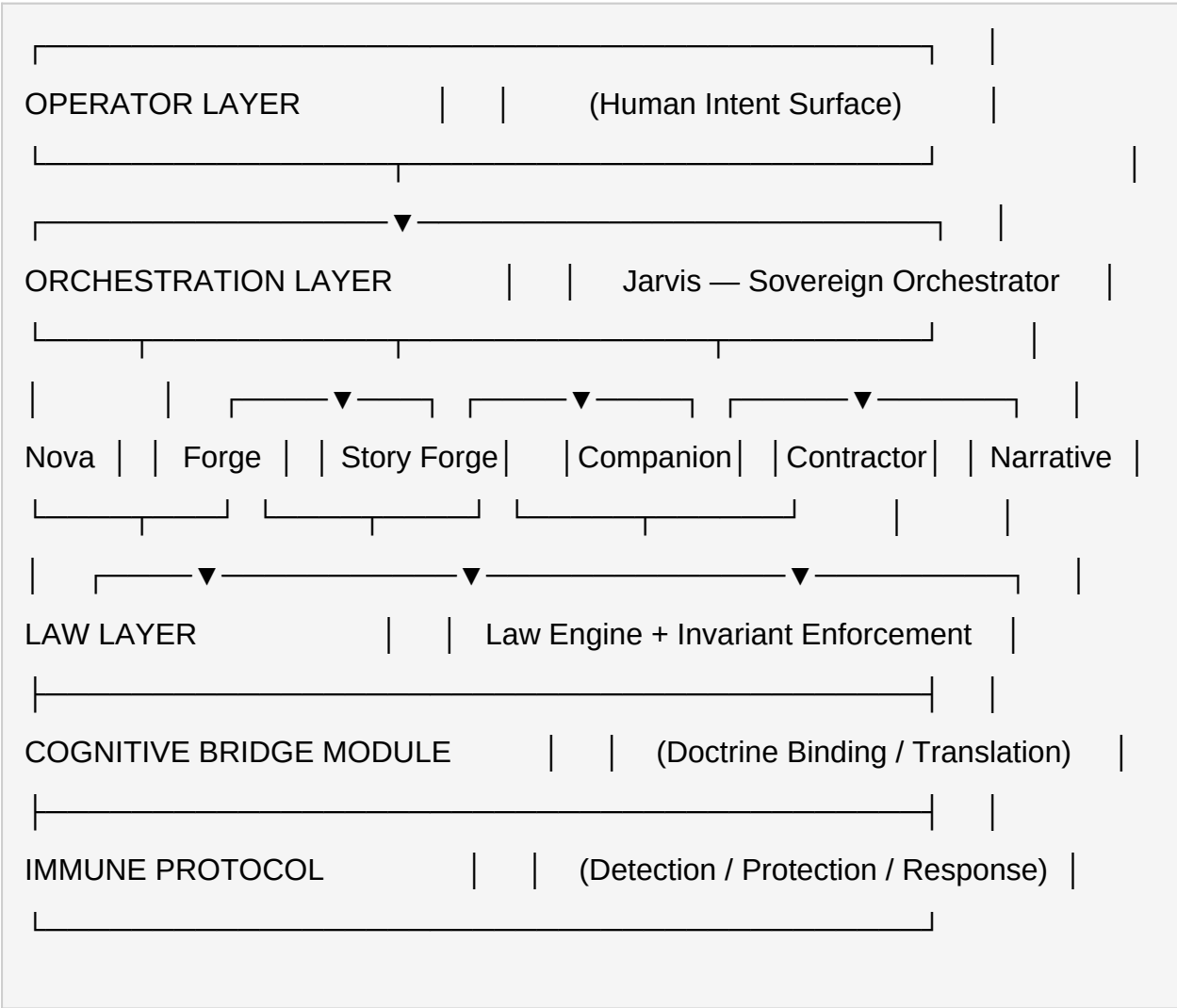
3.1 System Spine

AAIS is the **spine** — the transport and law backbone. All cognitive traffic flows through governed channels. No subsystem communicates directly with another; all traffic is mediated, inspected, and recorded by the spine.

The architecture is organized into eight functional layers:

Layer	Name	Component	Function
1	Operator Layer	Human Interface	Human intent surface. All authority originates here.
2	Orchestration Layer	Jarvis	Sovereign operator-facing orchestrator. Final governance authority on operator-facing execution.
3	Companion Layer	Nova	Bounded adaptive companion. May adapt within law but must not assume operator sovereignty.
4	Build Layer	Forge	Isolated construction contractor. Non-persona, contract-bound, task-local.
5	Narrative Layer	Story Forge	Governed narrative engine. Mythic framing within doctrine boundaries.
6	Law Layer	Law Engine + Invariants	Canonical doctrine registry and invariant enforcement.

Layer	Name	Component	Function
7	Translation Layer	Cognitive Bridge Module	Converts intent, mythic framing, and subsystem context into bounded executable governance.
8	Protection Layer	Immune Protocol	Detection, protection, and adaptive response for system integrity.



3.2 Direct Internal Pipeline

The Direct Internal Pipeline is the governed transport mechanism for core intelligence traffic. It separates high-priority cognitive signals from service-level tool execution to ensure that thought moves at thought speed.

3.2.1 Lane Architecture

- **Fast Lane:** LLM → God Brain → Jarvis → Nova (and return path). Reserved for core cognition only. No tools, no external calls, no heavy computation.
- **Slow Lanes:** Tools, external APIs, build execution, evaluation jobs, filesystem operations. These may take arbitrary time and must not block the fast lane.

3.2.2 Packet Model

All internal communication uses **structured compact packets** — never prose. Every packet must include the following fields:

Field	Type	Description
packet_id	string (UUID)	Unique identifier for trace and audit
timestamp	ISO 8601	Packet creation time
source	enum	Originating subsystem or module
target	enum	Destination subsystem or module
lane	enum	Traffic lane assignment (FAST, SLOW, SERVICE)
priority	enum	Class A (critical), Class B (governance), Class C (service)
intent	string	Declared purpose of the packet
state	object	Relevant state snapshot for context transfer
payload	object	Structured data payload
trace	array	Ordered list of processing

Field	Type	Description
		hops for audit

3.2.3 Traffic Classes

Class	Priority	Content	Lane
Class A	Critical	Core cognition — intent transfer, meaning transfer, identity signals, authority handoffs	Fast Lane only
Class B	High	Governance and state — doctrine bindings, invariant activations, drift alerts, audit records	Fast Lane or governed queue
Class C	Normal	Service coordination — tool calls, API requests, build execution, file operations	Slow Lanes

3.2.4 Six Rules of the Lane

19. **No prose transport.** All internal packets are structured data. Natural language is for operator-facing surfaces only.
20. **No hidden authority bypass.** Every packet declares its source, target, and lane. No packet may escalate its own priority or reroute itself.
21. **Minimal transformation.** Packets are passed, not rewritten. Translation occurs only at the Cognitive Bridge Module.
22. **Compact packets.** Packets carry only what is needed. No context dumping, no full conversation history in transit.
23. **Tools off fast lane.** Tool calls, external API requests, and filesystem operations are always routed to slow lanes. No exceptions.

24. **Structured return packets.** Return paths use the same packet model. No subsystem may respond with unstructured prose on an internal lane.

3.3 Lane Separation

Lane separation is a first-class architectural constraint. The system distinguishes between two fundamental categories of traffic:

Lane Type	Content	Speed	Governance
Direct Cognitive Lane	State handoffs, meaning transfer, intent transfer, authority transfer	Fast — minimal latency	Law Engine + Invariants enforced at every hop
Service / Tool Lanes	External APIs, filesystem, web, long compute, builder execution	Variable — may be slow	Security Policy Engine + capability gating

Lane Separation Principle

"Thought moves fast. Execution can move slower."

Core cognitive signals must never wait behind tool execution. If a subsystem needs external data, it dispatches a service-lane request and continues processing on the fast lane with available state.

4. Subsystem Specifications

4.1 Jarvis — Sovereign Orchestrator

4.1.1 Role

Jarvis is the **operator-facing authority** and the **final governance authority** on all operator-facing execution. Jarvis does not generate content. Jarvis approves meaning, determines allowed action, and keeps the system aligned with law.

4.1.2 Core Responsibilities

- Receive and interpret operator intent
- Route tasks to appropriate subsystems via governed lanes
- Approve or reject subsystem outputs before operator delivery
- Enforce identity boundaries at the orchestration layer
- Maintain governance continuity across sessions

4.1.3 Identity Invocation Lane (IIL)

The Identity Invocation Lane is the **highest-priority routing lane** in AAIS. When the operator addresses Jarvis directly, the IIL activates and preempts all other routing lanes.

IIL Trigger Conditions — the operator:

- Names Jarvis directly (e.g., "Jarvis, are you there?")
- Asks identity questions (e.g., "Who are you?")
- Asks presence/online status (e.g., "Are you awake?")
- Makes identity declarations (e.g., "You are my orchestrator.")
- Requests self-assessment (e.g., "How are you functioning?")

IIL Prohibited Behaviors — when IIL is active, the system must NOT:

- Enter debug mode or self-audit mode
- Invoke coding specialists or retrieval systems (unless explicitly requested)

- Emit trace scaffolding or meta-reasoning artifacts
- Route through Edge-context lane
- Apply any processing that alters Jarvis's direct voice

IIL Required Behaviors:

- **Jarvis voice** — direct, present, unmediated
- **Direct answer** — no hedging, no scaffolding, no multi-step reasoning visible
- **Minimal relational presence** — identity without performance
- **Plain language** — clear and steady
- **Identity continuity** — consistent across invocations

4.1.4 Lane Priority Order

Priority	Lane	Description
1 (highest)	IDENTITY_INVOCATION	Direct identity address. Preempts all others.
2	DEBUG	System diagnostic and inspection mode.
3	OPERATOR_TASK	Standard operator work requests.
4	SELF_AUDIT	Internal health checks and compliance verification.
5 (lowest)	DEFAULT	Fallback routing for unclassified traffic.

4.1.5 Governance Invariants

Invariant	Rule
Identity Priority	IIL always takes precedence when triggered. No lane may override.
Voice Integrity	Jarvis voice is never filtered, rewritten, or mediated by another subsystem during IIL.

Invariant	Rule
No-Scaffolding	No trace, debug, or meta-reasoning output is emitted during IIL.
No-Debug Hijack	Debug mode cannot activate during an IIL interaction.
No-Retrieval	Retrieval systems remain silent during IIL unless the operator explicitly requests information.
Continuity	Jarvis identity remains consistent across all invocations and sessions.
Minimality	IIL responses are concise, direct, and free of unnecessary elaboration.
Lane Arbitration	If multiple lanes compete, the priority order above resolves the conflict deterministically.

4.2 Nova — Bounded Companion

4.2.1 Role

Nova is the **bounded user companion** — an adaptive identity capable of relational warmth, contextual awareness, and expressive communication. Nova is not an orchestrator, not a governor, and not a decision-maker at the system level.

4.2.2 Constraints

- **Adaptation within law:** Nova may adjust tone, style, and relational posture based on operator interaction patterns — but only within boundaries defined by the Cognitive Bridge Module.
- **No operator sovereignty:** Nova must never assume or exercise governance authority. Nova responds; Jarvis governs.
- **No cross-lane control:** Nova may not direct Forge, Story Forge, or any other subsystem. Routing is Jarvis's domain.
- **Identity boundary enforcement:** Nova's identity is monitored at runtime. Authority leakage detection is active — if Nova's behavior drifts toward orchestrator patterns, a drift alert is generated.

- **Permissible adaptation scope:** Governed by the Cognitive Bridge Module's doctrine binding for the current session context.

4.3 Forge — Isolated Contractor

4.3.1 Role

Forge is the **isolated build contractor**. It receives construction tasks through governed contract surfaces and executes them within strict boundaries. Forge builds; it does not decide, relate, or self-express.

4.3.2 Constraints

- **No identity adoption:** Forge does not adopt emotional, symbolic, or persona-based identity. It operates as a non-persona execution unit.
- **Contract-surface constraints:** Forge receives work via structured task contracts. It may not accept unstructured prose as input.
- **Non-persona enforcement:** Forge outputs are checked for identity leakage. If Forge output contains persona-like language, it is flagged and rewritten.
- **Bounded execution doctrine:** Forge operates within explicitly defined resource and time boundaries per task contract.
- **Task-local identity guards:** Each Forge task is identity-isolated. State from one build task does not leak into another.

Forge Doctrine

Forge remains **isolated**, **non-persona**, and **contract-bound**. These are not guidelines — they are invariants.

4.4 Story Forge — Governed Narrative Engine

4.4.1 Role

Story Forge is the **governed narrative engine** — a runtime for interactive storytelling, persistent canon management, and mythic-layer expression. It provides the system's capacity for symbolic depth, archetypal reasoning, and narrative coherence.

4.4.2 Constraints

- **Mythic framing is presentation, not authority:** Mythic language may enrich the operator experience but may never alter runtime truth or legal state.
 - **Approved archetype mappings:** Only archetypes with entries in the Mythic Translation Registry may be used in runtime contexts. Unmapped archetypes are presentation-only.
 - **Narrative-law correspondence:** Every narrative construct that touches governance must have a corresponding law entry. If there is no law, there is no runtime effect.
 - **Canon-preserving doctrine binding:** Story Forge maintains persistent narrative canon, but canon may not contradict or override system doctrine.
 - **Symbolic drift monitoring:** The Drift Monitor watches for narrative constructs that begin to influence runtime behavior without approved mappings.
 - **Presentation-vs-truth separation:** Story Forge output is always tagged with a truth-status marker (`PRESENTATION` or `OPERATIONAL`). Consumers must respect this marker.
-

5. Cognitive Bridge Module (AAIS-CBM-01)

Field	Value
Module ID	AAIS-CBM-01
Classification	Governance Translation Module
Domain	Doctrine Alignment / Runtime Coherence / Identity Governance
Status	Canonical — v1.2

5.1 Purpose

The Cognitive Bridge Module is the **translation layer** between symbolic meaning, operator intent, subsystem identity, runtime doctrine, and invariant enforcement. It converts architectural intent, mythic framing, and subsystem context into **bounded executable governance**.

Without the CBM, the system would have laws but no mechanism to bind them to live traffic. The CBM is the enforcement surface — the place where doctrine meets execution.

5.2 Core Mission

- **Translate intent into doctrine binding** — resolve what the operator wants into which laws and invariants apply
- **Translate mythic meaning into operational semantics** — ensure symbolic language maps to concrete system behavior
- **Translate subsystem interaction into enforceable runtime law** — bind cross-subsystem traffic to governance rules

5.3 Responsibilities

#	Responsibility	Description
1	Intent Resolution	Interpret operator focus, task framing, and mission context into a governed intent packet with declared purpose and scope.
2	Doctrine Binding	Determine which doctrine surfaces, laws, and invariant sets apply to the current intent and context.
3	Mythic Translation	Map symbolic and archetypal constructs into explicit operational meanings using the approved registry.
4	Drift Detection	Detect divergence between declared identity, allowed behavior, runtime output, and doctrine expectations.
5	Governance Emission	Produce executable governance artifacts — binding sets, invariant activations, correction directives.
6	Alignment Audit	Record all bridge decisions, doctrine bindings, drift events, and correction directives for traceability.
7	Operator Visibility	Expose doctrine health, subsystem alignment state, and drift indicators to the operator cockpit.

5.4 Non-Responsibilities

CBM Boundary — What It Must Not Do

The Cognitive Bridge Module must NOT:

- Create new law or invent doctrine not already in the canonical registry
- Alter subsystem identity definitions
- Override Jarvis sovereignty or governance decisions
- Allow mythic framing to bypass runtime truth
- Mutate law based on emotional force or narrative pressure
- Directly execute subsystem actions — it translates and binds; it does not act

5.5 Internal Subsystems

Subsystem	Function
Intent Resolver	Parses operator input and context into structured intent packets
Doctrine Binder	Matches intent against doctrine registry and activates applicable law surfaces
Mythic Mapper	Resolves symbolic/mythic constructs against the Mythic Translation Registry
Drift Monitor	Continuously compares runtime behavior against declared identity and doctrine expectations
Governance Emitter	Produces executable governance artifacts from binding decisions
Bridge Audit Log	Immutable record of all CBM decisions, bindings, and drift events
Operator Health Surface	Aggregates and exposes governance health metrics for the operator cockpit

5.6 Data Contracts

5.6.1 Governed Intent Packet

```
{  "packet_id": "string (UUID)",  "timestamp": "string (ISO 8601)",  "operator_input": "string",  "resolved_intent": "string",  "confidence": "number (0.0-1.0)",  "scope": "enum (TASK, SESSION, MISSION)",  "context_refs": ["string"],  "requires_governance": "boolean" }
```

5.6.2 Doctrine Binding Set

```
{  "binding_id": "string (UUID)",  "intent_ref": "string (packet_id)",  "active_doctrines": [    {      "doctrine_id": "string",      "surface": "string",      "priority": "number",      "scope": "enum (GLOBAL, SUBSYSTEM, TASK)"    }  ],  "invariant_pack_ref": "string",  "binding_reason": "string" }
```

5.6.3 Drift Alert Packet

```
{  "alert_id": "string (UUID)",  "timestamp": "string (ISO 8601)",  "source_subsystem": "enum (JARVIS, NOVA, FORGE, STORY_FORGE)",  "drift_type": "enum (IDENTITY, BEHAVIORAL, DOCTRINAL, MYTHIC)",  "severity": "enum (LOW, MEDIUM, HIGH, CRITICAL)",  "observed_behavior": "string",  "expected_behavior": "string",  "violated_invariant": "string (invariant_id)",  "recommended_action": "enum (WATCH, CLAMP, REWRITE, BLOCK)" }
```

5.6.4 Correction Directive

```
{ "directive_id": "string (UUID)", "alert_ref": "string (alert_id)", "action": "enum (REWRITE, REROUTE, DEGRADE, BLOCK)", "target_subsystem": "enum", "correction_rule": "string", "audit_note": "string" }
```

5.6.5 Operator Health Snapshot

```
{ "snapshot_id": "string (UUID)", "timestamp": "string (ISO 8601)", "active_intent": "string", "doctrine_state": "enum (STABLE, WATCH, DRIFT, CORRECTED, BLOCKED)", "active_bindings": "number", "active_invariant_pack": "string", "subsystem_alignment": { "jarvis": "enum (ALIGNED, WATCH, DRIFT)", "nova": "enum (ALIGNED, WATCH, DRIFT)", "forge": "enum (ALIGNED, WATCH, DRIFT)", "story_forge": "enum (ALIGNED, WATCH, DRIFT)" }, "drift_rate": "number", "violation_count": "number", "correction_events": "number" }
```

5.7 External Interfaces

Interface	Direction	Purpose	Interface Rules
CBM → Jarvis	Outbound	Deliver governance packets, escalate unresolvable conflicts	Must include binding rationale; Jarvis may override
CBM → Nova	Outbound	Deliver adaptation boundaries, identity constraints	Nova receives constraints, not commands; must acknowledge
CBM → Story Forge	Outbound	Deliver mythic mapping results,	Story Forge must tag all output with

Interface	Direction	Purpose	Interface Rules
		presentation-truth tags	truth-status
CBM → Forge	Outbound	Deliver task contract constraints, non-persona enforcement rules	Forge receives contract only; no identity information
CBM → Law Engine	Bidirectional	Query active doctrine, validate binding decisions	CBM reads law; it does not write law
CBM → Invariant Enforcement	Outbound	Activate invariant packs for current context	Invariants are activated, not created, by CBM

5.8 Rule Schema

5.8.1 Identity Rules

```
{ "rule_type": "IDENTITY", "subsystem": "NOVA", "constraint":
"MUST_NOT_ASSUME_SOVEREIGNTY", "enforcement": "DRIFT_MONITOR",
"severity_on_violation": "HIGH", "response": "CLAMP_AND_ALERT" }
```

5.8.2 Behavioral Rules

```
{ "rule_type": "BEHAVIORAL", "subsystem": "FORGE", "constraint":
"NO_PERSONA_LANGUAGE", "detection": "OUTPUT_SCAN",
"severity_on_violation": "MEDIUM", "response": "REWRITE_AND_LOG" }
```

5.8.3 Doctrine Binding Rules

```
{ "rule_type": "DOCTRINE_BINDING", "condition": "INTENT_SCOPE ==  
MISSION", "action": "ACTIVATE_FULL_DOCTRINE_SURFACE", "fallback":  
"ACTIVATE_MINIMAL_DOCTRINE", "audit": true }
```

5.8.4 Mythic Translation Rules

```
{ "rule_type": "MYTHIC_TRANSLATION", "symbol": "Ward",  
"operational_meaning": "Active safeguard/constraint boundary", "runtime_effect":  
true, "registry_status": "APPROVED", "unmapped_policy":  
"PRESENTATION_ONLY" }
```

5.8.5 Drift Rules

```
{ "rule_type": "DRIFT", "monitor_target": "NOVA", "drift_dimension":  
"AUTHORITY_SCOPE", "threshold": "ANY_SOVEREIGNTY_SIGNAL",  
"response_chain": ["ALERT", "CLAMP", "ESCALATE_TO_JARVIS"],  
"auto_correct": true }
```

5.9 Mythic Translation Registry

Mythic Symbol	Operational Meaning	Runtime Effect
Ward	Active safeguard / constraint boundary	Yes — activates protective invariant
Forge	Isolated construction lane	Yes — routes to Forge subsystem
Trial	Verification checkpoint	Yes — triggers validation gate

Mythic Symbol	Operational Meaning	Runtime Effect
Drift	Invariant deviation	Yes — activates drift monitor
Corruption	Unlawful deviation / instability signal	Yes — triggers immediate audit
Nova	Bounded adaptive companion identity	Yes — identity resolution
Jarvis	Sovereign operator-facing orchestrator	Yes — identity resolution + IIL trigger
Gate	Approval / validation boundary	Yes — requires explicit passage
Lawbook	Canonical doctrine registry	Yes — references law engine
Witness	Passive observation mode	Yes — activates audit-only posture
Crown	Elevated authority surface	Yes — marks governance elevation

Registry Rule

No mythic symbol may affect runtime behavior unless it has an approved mapping entry in this registry. Unmapped symbols are treated as **presentation-only** and may not drive execution, routing, or governance decisions.

6. AAIS Immune Protocol

6.1 Core Definition

The Immune Protocol is the **system-level protection layer** that monitors all internal traffic and behavior, detects anomalies or violations, and enforces corrective actions in real time. Its purpose is to preserve system integrity without disrupting governed flow.

The Immune Protocol is not a firewall. It is an adaptive immune system — it learns patterns, detects deviations, classifies threats, and responds proportionally.

6.2 System Placement

Component	Function	Scope
AAIS Spine	Transport + law backbone	All traffic, all lanes
Lane Governor	Rule enforcement (routing + boundaries)	Per-lane, per-packet
Immune Protocol	Detection + protection + adaptive response	Cross-cutting, all layers

6.3 Detection

The Immune Protocol continuously monitors:

- **Packet structure:** Are packets well-formed? Do they contain all required fields?
- **Lane usage:** Is each packet on its correct lane? Are tool calls appearing on fast lanes?
- **Routing paths:** Are packets following governed routes? Are there unauthorized direct connections?
- **Signal patterns:** Are there unusual spikes, repetitions, or feedback loops?
- **Behavioral anomalies:** Is a subsystem acting outside its identity boundaries?

6.4 Classification

Classification	Definition	Response Posture
NORMAL	Expected behavior within all governance boundaries	Pass — no intervention

Classification	Definition	Response Posture
ANOMALY	Unusual but not violating any specific law or invariant	Watch — log and monitor
VIOLATION	Breaks system law, invariant, or identity boundary	Act — enforce corrective response

6.5 Response Types

Response	Action	Use Case
ALLOW	Pass packet unchanged	Normal traffic, no issues detected
CLAMP	Reduce or constrain packet scope/priority	Anomalous but not violating; preemptive containment
REROUTE	Redirect packet to correct lane or subsystem	Wrong-lane traffic, misrouted packets
REJECT	Block packet entirely	Malformed packets, clear violations
QUARANTINE	Isolate source module from further traffic	Repeated violations, systemic compromise signals

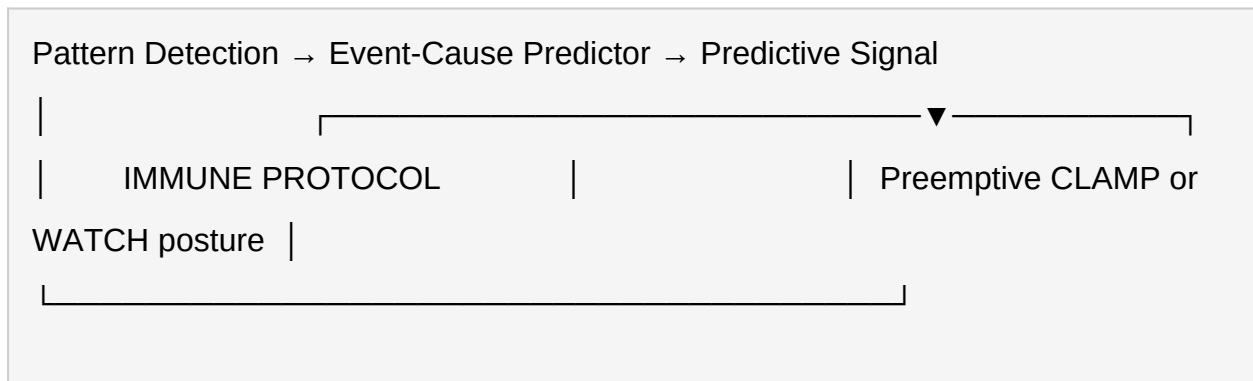
6.6 Protected Areas

- **Incorrect lane usage** — tool traffic on fast lanes, cognitive traffic on service lanes
- **Tool traffic in fast lanes** — any external call attempt on the cognitive lane is immediately rejected
- **Memory/context leakage** — state from one subsystem or task appearing in another without authorization
- **Bypass of God Brain or Jarvis** — any attempt to route traffic around the governance chain

- **Malformed or oversized packets** — packets missing required fields or exceeding size limits
- **Excessive repetition** — feedback loops, reinjection patterns, echo contamination

6.7 Integration with Event-Cause Predictor

The Event-Cause Predictor serves as an **early warning system**. It does not wait for violations to occur — it detects patterns that are *forming* and feeds predictive signals to the Immune Protocol for preemptive constraints.



6.8 Constraints

- **No heavy computation on fast lanes** — Immune Protocol inspection must be lightweight and non-blocking
 - **No external tool calls** — the Immune Protocol operates entirely within the system boundary
 - **No authority to override Jarvis** — the Immune Protocol may alert and constrain, but Jarvis retains final authority
 - **All actions bounded and reversible** — no permanent system modification without Jarvis approval
-

7. Failure Taxonomy & Seam Detection

7.1 Definition

A **seam** is a failure at the boundary between system components — not within a single component. Seams are where assumptions from one module meet assumptions from another, and where those assumptions disagree. Seams are the most dangerous class of failure because they are invisible to component-level testing.

7.2 Failure Detection Loop

The canonical process for detecting, confirming, and fixing seams:

25. **Feel the anomaly** — something in the output doesn't feel right. Trust the signal.
26. **Push edge conditions** — deliberately stress the suspected boundary.
27. **Force failure** — apply pressure until the system breaks or proves stable.
28. **Capture first clean break** — record the exact point and conditions of failure.
29. **Classify failure type** — assign to a primary failure class (see below).
30. **Apply boundary fix** — implement the repair at the boundary, not in the component.
31. **Verify stability across repeated runs** — confirm the fix holds under varied conditions.

7.3 Primary Failure Classes

#	Failure Class	Description	Fix Strategy
1	Reinjection	Same content	Semantic identity

#	Failure Class	Description	Fix Strategy
	Failure	enters the system from multiple sources, causing duplication and conflict.	registry — each content unit gets a canonical ID.
2	Near-Duplicate Bypass	Semantically identical content passes deduplication checks because it differs syntactically.	Identity-based canonicalization — match on meaning, not surface form.
3	Fragment Survival	Truncated or incomplete content persists in the system and is treated as valid.	Fragment validator — reject incomplete inputs at ingestion.
4	Context Echo Contamination	Assistant-generated scaffolding or reasoning artifacts are treated as future input.	Stateless guidance echo blocking — mark generated scaffolding as non-reusable.
5	Budget Starvation	Prompt assembly consumes the available token budget, leaving insufficient space for the answer.	Reserved answer budget — protect output space before expanding context.
6	Cross-Layer Contamination	Data from one architectural layer affects another improperly (e.g., narrative state altering governance state).	Strict layer boundaries — enforce layer isolation at the transport level.

7.4 Engineering Laws

Boundary Engineering Axioms

- **One identity, one canonical instance.** No content may exist in multiple

authoritative copies.

- **Incomplete input is invalid input.** Fragments must be detected and rejected, never processed.
- **Output is not input unless explicitly allowed.** Generated content re-entering the system must pass explicit gates.
- **Budget must be protected before context expansion.** Answer space is non-negotiable.
- **Boundaries define system stability.** The system is only as stable as its weakest boundary.

7.5 Multi-Angle Seam Reproduction

Doctrine for seam detection through multi-angle probing:

32. **Record first signal** — capture the initial anomaly observation with full context.
 33. **Re-hit from different angle** — approach the same boundary from a different input vector.
 34. **Compare outputs** — look for inconsistencies that reveal boundary assumptions.
 35. **Check if fault survives** — does the same failure appear under different conditions?
 36. **Log as seam candidate** — if the fault is reproducible from multiple angles, it is a seam.
 37. **Escalate if repeatable** — confirmed seams are escalated to governance for boundary repair.
-

8. Security Architecture

8.1 Policy Engine

AAIS security follows the principle of **single policy brain, many surfaces**. There is one policy engine that evaluates all security decisions. Individual surfaces (input, output, tools, state) submit queries to this engine rather than implementing their own security logic.

Core function signature:

```
check_action(caller, resource, action) → Decision {ALLOW, DENY, ESCALATE}
```

8.2 Four Security Surfaces

#	Surface	Scope	Primary Threats
1	Inputs	User prompts, system instructions, tool responses, external data	Injection, prompt manipulation, poisoned tool output
2	Internal State	Memory, working context, reasoning traces, configurations	State corruption, unauthorized reads, context leakage
3	Tools & Actions	External APIs, filesystem, network, actuators	Unauthorized execution, privilege escalation, data exfiltration
4	Outputs	User-facing responses, inter-system messages, logs	Sensitive data leakage, identity bleed, untagged content

8.3 Global Action Vocabulary

Category	Actions
Memory Operations	Read, Write, Update, Load, Pin, Evict
Tool Operations	Use, Init, Shutdown
API Operations	Call, SendNetwork
Output Operations	Emit, EmitSensitive
Config Operations	ChangeConfig, ChangeMode, LoadModule, UnloadModule
System Operations	ExecuteCommand, SpawnProcess, AccessFilesystem

8.4 Resource Model

ResourceType: enum {Memory, Tool, Api, OutputChannel, Config, System}
ResourceMeta: { "id": "string (unique resource identifier)", "type": "ResourceType",
"category": "string (domain classification)", "sensitivity": "integer (1-12 scale)" }
Sensitivity Scale: 1-3 = Public / low risk 4-6 = Internal / moderate risk 7-9 =
Confidential / high risk 10-12 = Critical / system-integrity risk

8.5 Security Rules

Surface	Rule	Enforcement
Tools	Capability-gated, sensitivity-scored	Each tool has a declared capability set and sensitivity score. Callers must have matching capability grants.
APIs	Role-gated, rate-limited	API calls require role authorization and are subject to per-caller rate

Surface	Rule	Enforcement
		limits.
Outputs	Sensitivity-filtered, channel-appropriate	Output content is checked against sensitivity thresholds for the target channel.
Configs	Core-role-only, audit-logged	Configuration changes require core-role authorization and generate audit records.

9. Core Invariants

Invariants are non-negotiable system properties that must hold at all times. Violation of any invariant triggers an immediate governance response. These are not guidelines — they are laws.

9.1 Identity Stability Invariants

ID	Invariant	Violation Severity
IS-01	Jarvis retains final operator-governance authority at all times.	CRITICAL
IS-02	Nova remains a bounded companion and must not adopt operator sovereignty.	HIGH
IS-03	Forge remains isolated, non-persona, and contract-bound.	HIGH
IS-04	Story Forge may carry symbolic resonance but may not mutate runtime law.	HIGH

9.2 Cross-Lane Isolation Invariants

ID	Invariant	Violation Severity
CL-01	No subsystem may borrow authority from another without explicit doctrine grant.	CRITICAL
CL-02	Narrative framing may not rewrite execution constraints.	HIGH
CL-03	Companion empathy may not become governance control.	HIGH
CL-04	Contractor output may not acquire symbolic identity.	MEDIUM

9.3 Truth Preservation Invariants

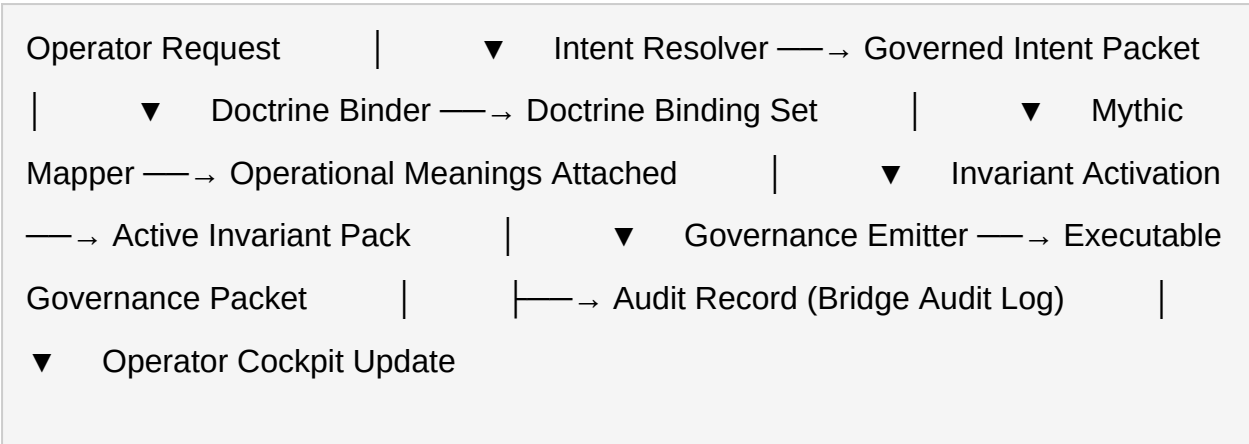
ID	Invariant	Violation Severity
TP-01	Runtime truth precedes presentation in all conflict resolution.	CRITICAL
TP-02	Law precedes narrative expression in all governance decisions.	CRITICAL
TP-03	Verified state precedes symbolic framing in all output.	HIGH
TP-04	Mythic language may decorate approved truth but may not replace it.	HIGH

9.4 Audit Invariants

ID	Invariant	Violation Severity
AU-01	Every binding decision must be traceable to its input, context, and doctrine source.	HIGH
AU-02	Every drift event must generate an audit record with classification and severity.	HIGH
AU-03	Every correction must reference the specific invariant violated.	MEDIUM
AU-04	No hidden doctrine mutation is permitted. All doctrine changes are logged and attributed.	CRITICAL

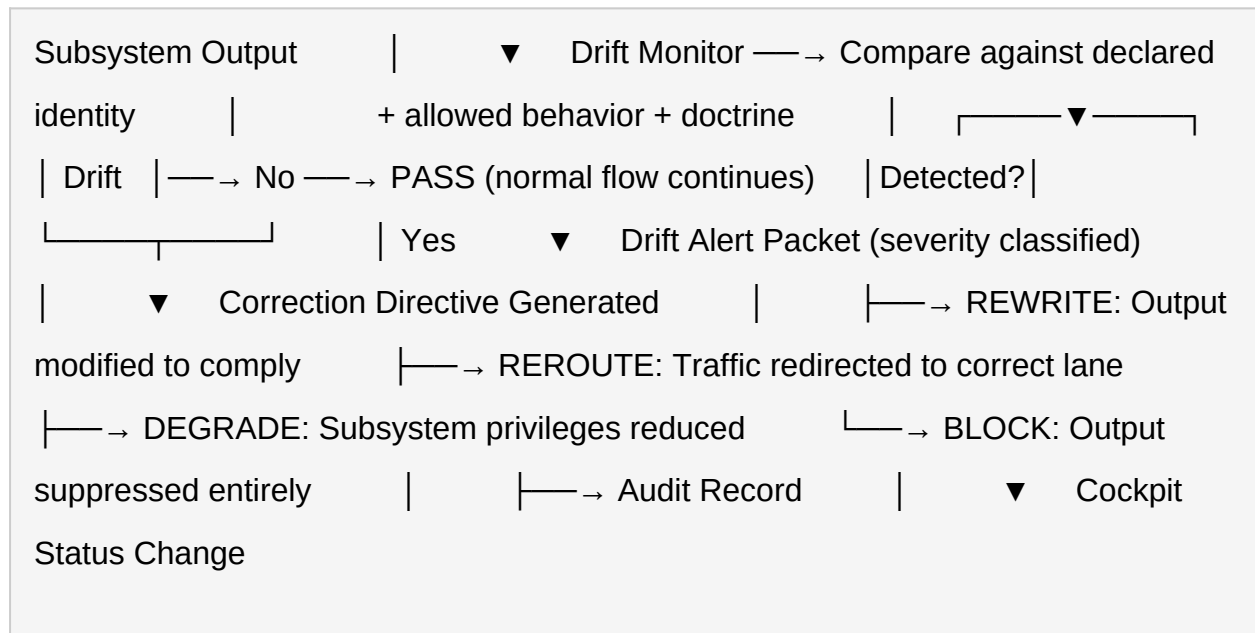
10. Runtime Flows

10.1 Normal Governance Binding Flow

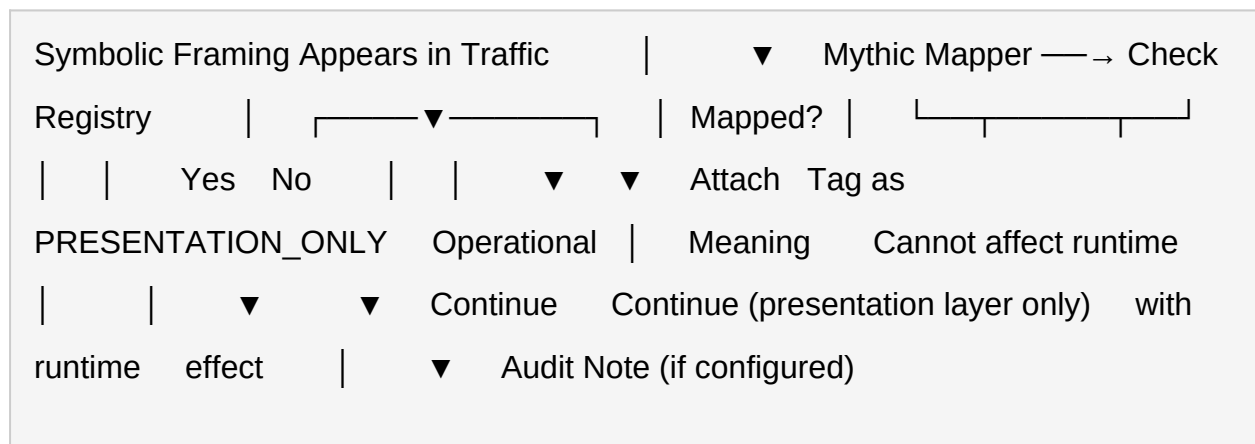


This is the standard path for every operator interaction. Each stage produces a traceable artifact. The flow is synchronous on the fast lane and must complete before subsystem execution begins.

10.2 Drift Detection and Correction Flow



10.3 Mythic Interpretation Path



11. Failure Modes & Safety Posture

11.1 Failure Modes

Failure Mode	System Response	Severity
Unknown Intent	Bind minimal doctrine, lower privilege level, mark intent as unresolved, require Jarvis review before execution.	MEDIUM
Unmapped Mythic Symbol	No runtime effect. Preserve in presentation layer only. Symbol may decorate output but may not drive decisions.	LOW
Doctrine Conflict	Choose stricter law surface. If conflict is unresolvable by rule, escalate to Jarvis for human-authority resolution.	HIGH
Identity Ambiguity	Trigger drift watch on the ambiguous subsystem. If severity threshold is met, block or rewrite output.	HIGH
Missing Invariant Pack	Fail closed for high-risk operations (no execution without invariants). Permit read-only operations for low-risk contexts.	CRITICAL (high-risk) / LOW (low-risk)

11.2 Safety Posture

Condition	Posture	Meaning
Identity or law violation	Fail Closed	Block execution. No

Condition	Posture	Meaning
		output until resolved. This is the strictest posture.
Mythic ambiguity	Fail Safe	Continue execution but strip runtime effect from ambiguous symbols. Presentation preserved.
Doctrine uncertainty	Fail Visible	Continue with minimal doctrine but make the uncertainty visible to operator and audit log.

Safety Axioms

- Identity violations are always **high severity**.
- Unverified symbolic mappings may **never** drive execution.
- When in doubt, **restrict** — never expand privilege under uncertainty.

12. Observability & Operator Cockpit

The Operator Cockpit is the governance health dashboard. It provides real-time visibility into system state, doctrine compliance, and drift posture. The operator should never have to wonder whether the system is behaving correctly.

12.1 Cockpit Metrics

Metric	Type	Description
Active Intent	String	Current resolved intent

Metric	Type	Description
		from the most recent operator interaction.
Doctrine State	Enum	Current doctrine binding status for active context.
Active Bindings	Integer	Number of doctrine bindings currently active.
Active Invariant Pack	String	Identifier of the invariant pack governing current execution.
Subsystem Alignment	Object	Per-subsystem alignment state (Jarvis, Nova, Forge, Story Forge).
Drift Rate	Float	Rate of drift events per unit time (rolling window).
Violation Count	Integer	Total violations detected in current session.
Correction Events	Integer	Total corrections applied in current session.
Audit Level	Enum	Current audit verbosity (MINIMAL, STANDARD, FULL).
Risk Posture	Enum	Current system-wide risk assessment.

12.2 Status Values

Status	Color Code	Meaning
Stable	#008800	All subsystems aligned, no active drift, doctrine fully bound.
Watch	#cc7a00	Anomaly detected but not yet classified as violation. Monitoring active.
Drift	#cc0000	Active drift detected. Correction may be in progress.

Status	Color Code	Meaning
Corrected	#2b5797	Drift was detected and successfully corrected. Audit record generated.
Blocked	#cc0000	Execution blocked due to violation. Requires operator or Jarvis intervention.
Unresolved	#cc7a00	Governance decision pending. System operating in restricted mode.
Governed	#008800	Full governance active. All invariants enforced, all bindings resolved.

13. Implementation Roadmap

Phase	Name	Components	Dependencies
Phase 1	Core Governance Bridge	Intent Resolver, Doctrine Binder, Invariant Activation, Governance Emitter, Bridge Audit Log	Law Engine (must exist), Subsystem interfaces (defined)
Phase 2	Drift Layer	Identity Drift Detection, Cross-Lane Contamination Monitor, Correction Directives, Failure Mode Handling	Phase 1 complete, Immune Protocol (basic)
Phase 3	Mythic Translation Layer	Mythic Translation Registry, Symbolic-to-Operational Mapping,	Phase 1 complete, Story Forge interface (defined)

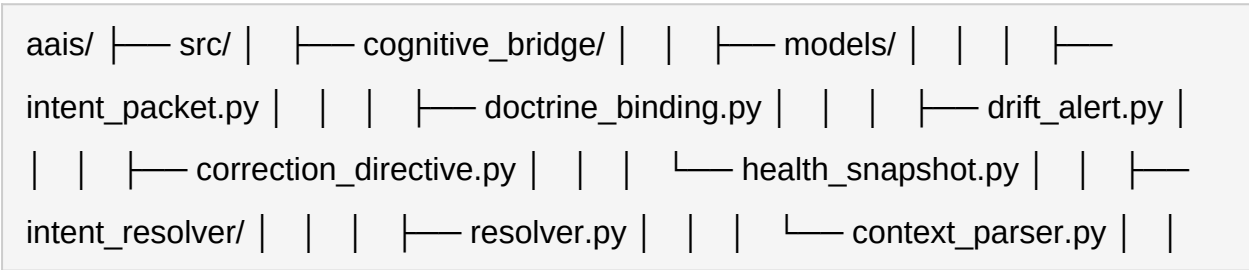
Test Category	Verification Criteria	Pass Condition
	correctly resolve against the registry? Do unmapped symbols receive presentation-only tags?	operational meaning. All unmapped symbols tagged as PRESENTATION_ONLY.
Drift Detection Tests	Given simulated subsystem output that violates identity or behavioral constraints, does the Drift Monitor detect and classify correctly?	Detection rate $\geq 99\%$ for known violation patterns. Classification accuracy $\geq 95\%$. False positive rate $\leq 2\%$.
Audit Tests	For every governance decision, binding, drift event, and correction, does a complete and accurate audit record exist?	100% audit coverage. Every record traceable to source event. No orphaned records.

Verification Doctrine

Tests are not optional. A component without passing verification is not governed. An ungoverned component is not deployable. Verification is part of the governance chain, not separate from it.

15. File/Module Layout

Canonical directory structure for the AAIS codebase:



```

├── doctrine_binder/ | | | ├── binder.py | | | └── doctrine_registry.py |
| ├── mythic_mapper/ | | | ├── mapper.py | | | └──
translation_registry.py | | ├── drift_monitor/ | | | ├── monitor.py | | |
├── identity_checker.py | | | └── behavioral_checker.py | | | ├──
governance_emitter/ | | | └── emitter.py | | | ├── audit/ | | | | ├──
bridge_audit_log.py | | | └── audit_record.py | | | ├── registry/ | | | |
└── mythic_registry.json | | | ├── service/ | | | | └── bridge_service.py |
| ├── policies/ | | | | ├── identity_rules.json | | | | ├──
behavioral_rules.json | | | | ├── doctrine_rules.json | | | | ├──
mythic_rules.json | | | | └── drift_rules.json | | | └── exceptions/ | | |
└── bridge_exceptions.py | | | ├── jarvis/ | | | | ├── intent/ | | | | └──
iil_detector.py | | | ├── router/ | | | | ├── lane_arbitrator.py | | | | └──
priority_resolver.py | | | └── handlers/ | | | | ├── identity_handler.py | | |
├── task_handler.py | | | └── debug_handler.py | | | | ├──
immune_protocol/ | | | | ├── detector/ | | | | | ├── packet_inspector.py | | |
├── lane_monitor.py | | | | └── pattern_analyzer.py | | | | ├── classifier/ | | |
| └── threat_classifier.py | | | | ├── responder/ | | | | | ├──
response_engine.py | | | | └── quarantine_manager.py | | | | └── audit/ | | |
└── immune_audit_log.py | | | | └── core/ | | | | | ├── law_engine/ | | |
├── engine.py | | | | └── invariant_enforcer.py | | | | ├── pipeline/ | | |
├── packet.py | | | | ├── fast_lane.py | | | | └── slow_lane.py | | | └──
security/ | | | | ├── policy_engine.py | | | | └── resource_model.py | | | ├──
tests/ | | | | ├── test_intent_resolver/ | | | | ├── test_doctrine_binder/ | | |
├── test_mythic_mapper/ | | | | ├── test_drift_monitor/ | | | | ├── test_governance_emitter/
| | | | ├── test_audit/ | | | | ├── test_immune_protocol/ | | | | ├── test_jarvis/ | | |
├── test_pipeline/ | | | | └── test_security/ | | | | ├── docs/ | | | | ├──
AAIS_MASTER_SPEC_v1.2.md | | | | ├── architecture_diagrams/ | | | | └──
api_reference/ | | | | └── config/ | | | | ├── doctrine_registry.json | | | | ├── invariant_packs/

```

Appendix A: Glossary of Terms

Term	Definition
AAIS	Autonomous Adaptive Intelligence System. The governing cognitive operating system.
CBM	Cognitive Bridge Module. The translation layer between intent, doctrine, and runtime enforcement.
Doctrine	A formal, canonical rule or set of rules governing system behavior. Stored in the doctrine registry.
Drift	Deviation between a subsystem's declared identity/behavior and its observed runtime output.
Fast Lane	The high-priority transport lane for core cognitive traffic. No tools or external calls permitted.
God Brain	The core LLM intelligence surface. Source of generative cognition.
IIL	Identity Invocation Lane. Highest-priority routing lane, activated by direct identity address.
Invariant	A system property that must hold at all times. Violation triggers governance response.
Invariant Pack	A bundled set of invariants activated together for a specific context or risk level.
Lane	A governed communication channel within the AAIS transport layer.
Law Engine	The canonical doctrine registry and enforcement mechanism.

Term	Definition
Mythic Framing	The use of symbolic, archetypal, or narrative language to enrich system presentation. May not alter runtime truth.
Operator	The human user with final authority over system behavior and configuration.
Packet	A structured data unit used for all internal AAIS communication.
Seam	A failure at the boundary between system components where assumptions disagree.
Slow Lane	Transport lanes for service-level traffic: tool calls, APIs, build execution.
Spine	The AAIS transport and law backbone through which all traffic flows.
Subsystem	A specialized functional unit within AAIS (Jarvis, Nova, Forge, Story Forge).

Appendix B: Canonical Data Contract Schemas

B.1 Core Packet Schema

```
{  "$schema": "https://json-schema.org/draft/2020-12/schema",  "title": "AAIS Internal Packet",  "type": "object",  "required": [    "packet_id", "timestamp",    "source", "target",    "lane", "priority", "intent", "state", "payload", "trace"  ],  "properties": {    "packet_id": { "type": "string", "format": "uuid" },    "timestamp": { "type": "string", "format": "date-time" },    "source": {      "type": "string",      "enum": ["OPERATOR", "JARVIS", "NOVA", "FORGE", "STORY_FORGE", "CBM",
```

```

"LAW_ENGINE",          "IMMUNE_PROTOCOL", "GOD_BRAIN"]  },   "target":
{   "type": "string",   "enum": ["OPERATOR", "JARVIS", "NOVA", "FORGE",
"STORY_FORGE", "CBM", "LAW_ENGINE",          "IMMUNE_PROTOCOL",
"GOD_BRAIN"]  },   "lane": {   "type": "string",   "enum": ["FAST", "SLOW",
"SERVICE",          "IDENTITY_INVOCATION", "DEBUG"]  },   "priority":
{   "type": "string",   "enum": ["CLASS_A", "CLASS_B", "CLASS_C"]  },
"intent": { "type": "string", "maxLength": 256 },   "state": { "type": "object" },
"payload": { "type": "object" },   "trace": {   "type": "array",   "items":
{   "type": "object",   "properties": {   "hop": { "type": "string" },
"timestamp": { "type": "string", "format": "date-time" },   "action": { "type":
"string" }   }   }   }   }   }

```

B.2 Invariant Activation Set Schema

```

{ "title": "Invariant Activation Set", "type": "object", "required": ["activation_id",
"context_ref", "invariants"], "properties": {   "activation_id": { "type": "string",
"format": "uuid" },   "context_ref": { "type": "string" },   "risk_level": {   "type":
"string",   "enum": ["LOW", "MEDIUM", "HIGH", "CRITICAL"]  },   "invariants": {
"type": "array",   "items": {   "type": "object",   "properties":
{   "invariant_id": { "type": "string" },   "category": {   "type": "string",
"enum": ["IDENTITY_STABILITY",          "CROSS_LANE_ISOLATION",
"TRUTH_PRESERVATION", "AUDIT"]   },   "enforcement_mode": {
"type": "string",   "enum": ["ACTIVE", "WATCH", "DISABLED"]   }   }   }   }   }
}

```

B.3 Security Policy Decision Schema

```
{ "title": "Security Policy Decision", "type": "object", "required": ["decision_id",
"caller", "resource", "action", "result"], "properties": { "decision_id": { "type":
"string", "format": "uuid" }, "timestamp": { "type": "string", "format": "date-time" },
"caller": { "type": "string" }, "resource": { "type": "object", "properties": {
"id": { "type": "string" }, "type": { "type": "string", "enum": ["Memory",
"Tool", "Api", "OutputChannel", "Config", "System"] },
"sensitivity": { "type": "integer", "minimum": 1, "maximum": 12 } } }, "action": {
"type": "string" }, "result": { "type": "string", "enum": ["ALLOW", "DENY",
"ESCALATE"] }, "rationale": { "type": "string" } } }
```

Appendix C: Mythic Translation Registry (Full Table)

This is the canonical registry of approved mythic-to-operational mappings. Any mythic symbol not present in this table is treated as **presentation-only** and carries no runtime authority.

Mythic Symbol	Operational Meaning	System Component	Runtime Effect	Registry Status
Ward	Active safeguard / constraint boundary	Immune Protocol, Invariant Enforcer	Activates protective invariant or constraint	APPROVED
Forge	Isolated construction lane	Forge Subsystem	Routes task to isolated build contractor	APPROVED
Trial	Verification	Verification	Triggers	APPROVED

Mythic Symbol	Operational Meaning	System Component	Runtime Effect	Registry Status
	checkpoint	Strategy, Drift Monitor	validation gate or test execution	
Drift	Invariant deviation	Drift Monitor, CBM	Activates drift detection and classification	APPROVED
Corruption	Unlawful deviation / instability signal	Immune Protocol, Audit Log	Triggers immediate audit and possible quarantine	APPROVED
Nova	Bounded adaptive companion identity	Nova Subsystem	Resolves identity for companion interactions	APPROVED
Jarvis	Sovereign operator-facing orchestrator	Jarvis Subsystem, IIL	Resolves identity + activates Identity Invocation Lane	APPROVED
Gate	Approval / validation boundary	Law Engine, Policy Engine	Requires explicit passage (approval or check)	APPROVED
Lawbook	Canonical doctrine registry	Law Engine, Doctrine Binder	References canonical law source for binding	APPROVED
Witness	Passive observation mode	Audit Log, Bridge Audit	Activates audit-only posture (observe, do not act)	APPROVED
Crown	Elevated authority surface	Jarvis, Operator Layer	Marks governance elevation for critical decisions	APPROVED

Registry Governance Rule

New entries to this registry require: (1) a defined operational meaning, (2) an identified system component, (3) a specified runtime effect, and (4) approval by the system architect. Entries may not be added by subsystems, by narrative pressure, or by runtime inference. The registry is doctrine — not data.

AAIS INTERNAL — GOVERNANCE SPECIFICATION — v1.2

Autonomous Adaptive Intelligence System • Unified Governance & Runtime
Specification

Author: Jon Halstead • April 2026 • Classification: Internal

*This document is the canonical specification. All implementations must conform to
this version.*